2025.8.19 FedAvg优化

> ⚠️ 每个最好的结果记录时不要忘记**截图**、**代码**和输入的**命令行参数**
>
> 得到一个目前最好的结果，可以进一步改变参数，比如**全局轮数、本地轮数、frac比例**等
>
> 每种不同的方法都可以记录一下各自最好的结果

- **过拟合风险**：
  - CIFAR-100 的每类样本数量较少，ResNet 的高复杂性可能导致过拟合。
  - 解决方法：
    - 使用数据增强（如随机裁剪、翻转、颜色抖动等）。
    - 增加正则化（如 Dropout 或权重衰减）。
- **计算资源需求**：
  - ResNet 的深层结构需要更多的计算资源和训练时间。
  - 如果资源有限，可以选择较浅的 ResNet 变体（如 ResNet-18 或 ResNet-20）。

一、mnist数据集

1.iid

（1）基线

sgd优化器

```
python baseline_main.py --dataset mnist --model cnn --epochs 100 --lr 0.01 --optimizer sgd --momentum 0.9 --local_bs 64 --gpu 0
```

```
class CNNMnist(nn.Module): # 定义一个针对 MNIST 数据集的 CNN 模型
    def __init__(self, args): # 构造函数，接收命令行参数作为输入
        super(CNNMnist, self).__init__()
        # 第一个卷积层：输入通道数为 args.num_channels (MNIST 为 1)，输出通道数为 10，卷积核大小为 5x5
```

```
        self.conv1 = nn.Conv2d(args.num_channels, 10, kernel_size=5)
        # 第二个卷积层：输入通道数为 10，输出通道数为 20，卷积核大小为 5x5
        self.conv2 = nn.Conv2d(10, 20, kernel_size=5)
        # Dropout 层，作用于2D特征图
        self.conv2_drop = nn.Dropout2d()
        # 第一个全连接层：输入特征数为 320 (这个值取决于前面卷积和池化后的特征图大小)，输出特征数为 50
        self.fc1 = nn.Linear(320, 50)
        # 第二个全连接层 (输出层)：输入特征数为 50，输出特征数为 args.num_classes (MNIST 为 10)
        self.fc2 = nn.Linear(50, args.num_classes)

    def forward(self, x): # 定义前向传播逻辑
        # x → conv1 → max_pool2d (2×2) → relu
        x = F.relu(F.max_pool2d(self.conv1(x), 2))
        # x → conv2 → conv2_drop → max_pool2d (2×2) → relu
        x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))
        # 将特征图展平
        x = x.view(-1, x.shape[1]*x.shape[2]*x.shape[3])
        x = F.relu(self.fc1(x)) # 通过第一个全连接层并应用 ReLU
        x = F.dropout(x, training=self.training) # 应用 Dropout，training=self.training 确保只在训练时生效
        x = self.fc2(x) # 通过输出层
        return F.log_softmax(x, dim=1) # 返回 log_softmax 输出，常用于 NLLLoss 损失函数
```

```
Train loss: 4.518198611775747e-05
Validation loss: 0.021804
Validation loss did not improve. Patience: 50/50
Early stopping triggered after 70 epochs
 69%|                                        | 69/100 [55:12<24:48, 48.01s/it]
Test on 10000 samples
Test Accuracy: 99.50%
```

adam优化器

```
python baseline_main.py --dataset mnist --model cnn --epochs 100 --lr 0.001 --optimizer adam --local_bs 64 --gpu 0
```

```
Train loss: 0.07933188775089631
100%|                                                  | 100/100 [38:21<00:00, 23.02s/it]
Test on 10000 samples
Test Accuracy: 99.07%
```

（2）普通FedAvg

sgd优化器

```
python federated_main.py --dataset mnist --model cnn --epochs 100 --num_users 100 --frac 0.1 --local_ep 5 --local_bs 10 --lr 0.01 --optimizer sgd --momentum 0.9 --iid 1 --gpu 0
```

模型与基线相同

```
Avg Training Stats after 100 global rounds:
Training Loss : 0.2775894650932154
Train Accuracy: 98.33%

100%|                                                  | 100/100 [49:05<00:00, 27
100%|                                                  | 100/100 [49:05<00:00, 29
.45s/it]

 Results after 100 global rounds of training:
|---- Avg Train Accuracy: 98.33%
|---- Test Accuracy: 98.31%
```

adam优化器

```
python federated_main.py \
    --dataset mnist \
    --model cnn \
    --epochs 100 \
    --num_users 100 \
    --frac 0.1 \
    --local_ep 5 \
    --local_bs 10 \
    --lr 0.01 \
    --optimizer adam \
```

```
    --iid 1 \
    --gpu 0
```

```
Avg Training Stats after 100 global rounds:
Training Loss : 0.6703804306223983
Train Accuracy: 96.67%

100%|                                        | 100/100 [25:19<00:00, 15.19s/it]

 Results after 100 global rounds of training:
|---- Avg Train Accuracy: 96.67%
|---- Test Accuracy: 94.31%
```

（3）优化FedAvg

```
python federated_main_improved.py --dataset mnist --model cnn --epochs 100 --lr 0.01 --frac 0.1 --num_users 100 --local_ep 10 --local_bs 32 --optimizer adam --stopping_rounds 15 --gpu 0
```

```
class CNNMnist(nn.Module): # 基于ResNet思想的MNIST模型
    def __init__(self, args):
        super(CNNMnist, self).__init__()
        # 使用ResNet-18的简化版本，适配28x28的MNIST图像
        self.conv1 = nn.Conv2d(args.num_channels, 64, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(64)

        # ResNet blocks - 使用残差连接
        self.layer1 = self._make_layer(64, 64, 2, stride=1)
        self.layer2 = self._make_layer(64, 128, 2, stride=2)
        self.layer3 = self._make_layer(128, 256, 2, stride=2)
        self.layer4 = self._make_layer(256, 512, 2, stride=2)

        # 分类层
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(512, args.num_classes)
        self.dropout = nn.Dropout(0.5)
```

```
    def _make_layer(self, in_channels, out_channels, blocks, stride):
        layers = []
        # 第一个block可能需要下采样
        layers.append(ResBlock(in_channels, out_channels, stride))
        # 后续block保持尺寸
        for _ in range(1, blocks):
            layers.append(ResBlock(out_channels, out_channels, 1))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = F.relu(self.bn1(self.conv1(x)))

        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)

        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.dropout(x)
        x = self.fc(x)

        return F.log_softmax(x, dim=1)


class ResBlock(nn.Module):
    """残差块"""
    def __init__(self, in_channels, out_channels, stride=1):
        super(ResBlock, self).__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stri
de=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, st
ride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        # 快捷连接
```

```
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=strid
e, bias=False),
                nn.BatchNorm2d(out_channels)
            )

    def forward(self, x):
        identity = self.shortcut(x)

        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))

        out += identity
        out = F.relu(out)

        return out
```

```
Avg Training Stats after 50 global rounds:
Training Loss : 0.04701985295611448
Train Accuracy: 98.82%

100%|[redacted]| 50/50 [11:37<00:00, 13.95s/it]

 Results after 50 global rounds of training:
|---- Avg Train Accuracy: 98.82%
|---- Test Accuracy: 99.09%

 Total Run Time: 700.4891
```

2.non-iid

（1）普通FedAvg

sgd优化器

```
python federated_main.py --dataset mnist --model cnn --epochs 100 --nu
m_users 100 --frac 0.1 --local_ep 10 --local_bs 10 --lr 0.001 --optimizer sgd
--iid 0 --gpu 0
```

```
Avg Training Stats after 46 global rounds:
Training Loss : 0.053798211898697657
Train Accuracy: 9.00%

Early stopping triggered after 46 global rounds
Loaded best model weights for final testing.
 45%|          | 45/100 [57:57<1:10:50, 77.28s/it]

 Results after 46 global rounds of training:
|---- Avg Train Accuracy: 9.00%
|---- Test Accuracy: 27.58%

 Total Run Time: 3480.0439
```

adam优化器

```
python federated_main.py \
    --dataset mnist \
    --model cnn \
    --epochs 100 \
    --num_users 100 \
    --frac 0.1 \
    --local_ep 5 \
    --local_bs 10 \
    --lr 0.01 \
    --optimizer adam \
    --iid 0 \
    --gpu 0
```

```
Avg Training Stats after 100 global rounds:
Training Loss : 0.12197368648592397
Train Accuracy: 96.67%

100%|                                        | 100/100 [25:13<00:00, 15.14s/it]

 Results after 100 global rounds of training:
|---- Avg Train Accuracy: 96.67%
|---- Test Accuracy: 34.01%
```

（2）优化FedAvg

```
python federated_main_improved.py --dataset mnist --model cnn --epochs 100 --lr 0.001 --frac 0.1 --num_users 100 --local_ep 5 --local_bs 32 --optimizer sgd --stopping_rounds 20 --gpu 0 --iid 0
```

模型与iid的一样

```
Avg Training Stats after 100 global rounds:
Training Loss : 0.12666636764254108
Train Accuracy: 97.52%

100%|▉| 100/100 [37:04<00:00, 22.

 Results after 100 global rounds of training:
|---- Avg Train Accuracy: 97.52%
|---- Test Accuracy: 97.61%

 Total Run Time: 2227.2196
```

二、cifar10数据集

1.iid

（1）基线

sgd优化器

```
python baseline_main.py --dataset cifar --model cnn --epochs 100 --lr 0.05 --momentum 0.9 --local_bs 64 --num_classes 10 --num_channels 3 --gpu 0 --optimizer sgd --verbose 1
```

```
class CNNCifar(nn.Module): # 定义一个针对 CIFAR-10 数据集的 CNN 模型
    def __init__(self, args):
        super(CNNCifar, self).__init__()
```

```
        # CIFAR-10 图像通道数为 3
        self.conv1 = nn.Conv2d(3, 6, 5) # 输入通道 3, 输出通道 6, 5x5 核
        self.pool = nn.MaxPool2d(2, 2) # 2×2 最大池化，步长为 2
        self.conv2 = nn.Conv2d(6, 16, 5) # 输入通道 6, 输出通道 16, 5x5 核
        # 全连接层
        self.fc1 = nn.Linear(16 * 5 * 5, 120) # 输入特征数取决于卷积和池化后的尺寸
                                # CIFAR-10 图像 32x32 → conv1 (28×28) → pool (14×14) → conv2 (10×10) → pool (5×5)
                                # 所以是 16 (通道数) * 5 * 5
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, args.num_classes) # 输出层，类别数为 args.num_classes (CIFAR-10 为 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # conv1 → relu → pool
        x = self.pool(F.relu(self.conv2(x))) # conv2 → relu → pool
        x = x.view(-1, 16 * 5 * 5) # 展平
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return F.log_softmax(x, dim=1) # 返回 log_softmax 输出
```

```
Avg Training Stats after 100 global rounds:
Training Loss : 0.42798910042929217
Train Accuracy: 46.00%

100%|                                                  | 100/100 [1:22:44<00:00, 49.65s/it]

 Results after 100 global rounds of training:
|---- Avg Train Accuracy: 46.00%
|---- Test Accuracy: 48.17%
```

adam优化器

（2）普通FedAvg

```
python federated_main.py \
    --model cnn \
    --dataset cifar \
    --epochs 100 \
    --num_users 100 \
    --frac 0.25 \
    --local_ep 12 \
    --local_bs 32 \
    --lr 0.03 \
    --momentum 0.9 \
    --optimizer sgd \
    --iid 1 \
    --stopping_rounds 30 \
    --verbose 1 \
    --gpu 0
```

模型与基线相同

（3）优化FedAvg

```
python src/federated_main.py \
    --model cnn \
    --dataset cifar \
    --epochs 100 \
    --num_users 100 \
    --frac 0.25 \
    --local_ep 12 \
    --local_bs 32 \
    --lr 0.03 \
    --momentum 0.9 \
    --optimizer sgd \
    --iid 1 \
    --early_stop 1 \
    --stop_metric accuracy \
    --stopping_rounds 30 \
```

```
    --verbose 1 \
    --gpu cuda:0
```

```
class CNNCifarOptimal(nn.Module):
    def __init__(self, args):
        super(CNNCifarOptimal, self).__init__()

        # 第一个卷积块 - 保持输入特征
        self.conv1 = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
        )

        self.conv2 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),
            nn.Dropout(0.2),  # 减少dropout，避免loss下降困难
        )

        self.conv3 = nn.Sequential(
            nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),
```

```
            nn.Dropout(0.25),  # 减少dropout
        )

        self.conv4 = nn.Sequential(
            nn.Conv2d(256, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),
            nn.Dropout(0.3),  # 减少dropout
        )

        # 优化分类器 - 减少参数复杂度，提高收敛性
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.classifier = nn.Sequential(
            nn.Dropout(0.4),  # 减少dropout
            nn.Linear(512, 256),  # 进一步减少参数量
            nn.ReLU(inplace=True),
            nn.Dropout(0.2),  # 减少dropout
            nn.Linear(256, args.num_classes)
        )

        # 权重初始化
        self._initialize_weights()

    def _initialize_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='r
elu')
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1)
                nn.init.constant_(m.bias, 0)
            elif isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, 0, 0.01)
                nn.init.constant_(m.bias, 0)
```

```
    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = self.conv3(x)
        x = self.conv4(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x
```

-

2.non-iid

（1）普通FedAvg

```
python federated_main.py --dataset cifar --model cnn --epochs 100 --num_users 100 --frac 0.2 --local_ep 15 --local_bs 20 --lr 0.003 --optimizer sgd --iid 0 --gpu 0
```

模型与基线、iid相同

```
Avg Training Stats after 100 global rounds:
Training Loss : 0.2098459992642896
Train Accuracy: 16.00%

100%|██████████████████████████████████████| 100/100 [1:22:48<00:00, 49.68s/it]

 Results after 100 global rounds of training:
|---- Avg Train Accuracy: 16.00%
|---- Test Accuracy: 36.97%
```

```
python federated_main.py \
    --model cnn \
    --dataset cifar \
    --epochs 100 \
    --num_users 100 \
```

```
    --frac 0.25 \
    --local_ep 12 \
    --local_bs 32 \
    --lr 0.03 \
    --momentum 0.9 \
    --optimizer sgd \
    --iid 0 \
    --stopping_rounds 30 \
    --verbose 1 \
    --gpu 0
```

```
Avg Training Stats after 100 global rounds:
Training Loss : 0.09779776720006017
Train Accuracy: 8.00%

100%|██████████████████████████████| 100/100 [2:07:25<00:00, 76.46s/it]

 Results after 100 global rounds of training:
|---- Avg Train Accuracy: 8.00%
|---- Test Accuracy: 42.52%
```

（2）优化FedAvg

三、cifar100数据集

1.iid

（1）基线

sgd优化器

```
python baseline_main.py \
    --dataset cifar100 \
    --model cnn \
    --epochs 350 \
```

```
    --lr 0.035 \
    --optimizer sgd \
    --local_bs 64 \
    --num_classes 100 \
    --gpu 0 \
    --verbose 1
```

普通CNN模型

```
class CNNCifar100(nn.Module): # 定义一个针对 CIFAR-100 数据集的 CNN 模型
    def __init__(self, args):
        super(CNNCifar100, self).__init__()
        # CIFAR-100 图像通道数为 3，类别数为 100
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1) # 输入通道 3, 输出通道 32, 3x3 核
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1) # 输入通道 32, 输出通道 64, 3x3 核
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1) # 输入通道 64, 输出通道 128, 3x3 核
        self.pool = nn.MaxPool2d(2, 2) # 2×2 最大池化，步长为 2
        self.dropout = nn.Dropout(0.5) # Dropout层防止过拟合

        # 全连接层
        # CIFAR-100 图像 32x32 → conv1+pool (16×16) → conv2+pool (8×8) → conv3+pool (4×4)
        # 所以是 128 (通道数) * 4 * 4
        self.fc1 = nn.Linear(128 * 4 * 4, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, args.num_classes) # 输出层，CIFAR-100 有 100 个类别

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # conv1 → relu → pool
        x = self.pool(F.relu(self.conv2(x))) # conv2 → relu → pool
        x = self.pool(F.relu(self.conv3(x))) # conv3 → relu → pool
        x = x.view(-1, 128 * 4 * 4) # 展平
        x = self.dropout(F.relu(self.fc1(x)))
```

```
        x = self.dropout(F.relu(self.fc2(x)))
        x = self.fc3(x)
        return F.log_softmax(x, dim=1) # 返回 log_softmax 输出
```

```
Train loss: 0.5932932535515112
100%|                                        | 350/350 [1:04:30<00:00, 11.06s/it]
Test on 10000 samples
Test Accuracy: 41.80%
```

adam优化器

```
python baseline_main.py \
    --dataset cifar100 \
    --model cnn \
    --epochs 350 \
    --lr 0.035 \
    --optimizer adam \
    --local_bs 64 \
    --num_classes 100 \
    --gpu 0 \
    --verbose 1
```

（2）普通FedAvg

sgd优化器

```
python federated_main.py \
    --dataset cifar100 \
    --model cnn \
    --epochs 350 \
    --lr 0.035 \
    --num_users 40 \
    --frac 0.5 \
    --local_ep 8 \
    --local_bs 64 \
```

```
    --optimizer sgd \
    --iid 1 \
    --num_classes 100 \
    --gpu 0 \
    --verbose 1
```

模型和基线相同

```
Avg Training Stats after 350 global rounds:
Training Loss : 0.2160085893932032
Train Accuracy: 54.00%

100%|                                                  | 350/350 [1:42:04<00:00, 20.05s/i
100%|                                                  | 350/350 [1:42:04<00:00, 17.50s/i
t]

 Results after 350 global rounds of training:
|---- Avg Train Accuracy: 54.00%
|---- Test Accuracy: 16.25%
```

adam优化器

```
python federated_main.py \
    --dataset cifar100 \
    --model cnn \
    --epochs 350 \
    --lr 0.035 \
    --num_users 40 \
    --frac 0.5 \
    --local_ep 8 \
    --local_bs 64 \
    --optimizer adam \
    --iid 1 \
    --num_classes 100 \
    --gpu 0 \
    --verbose 1
```

（3）优化FedAvg

```
python federated_cifar100_main.py --model ultra --epochs 350 --num_users 40 --frac 0.5 --local_ep 10 --local_bs 64 --lr 0.004 --optimizer adamw --dropout_rate 0.3 --label_smoothing 0.1 --alpha 0.2 --gpu 0 --iid 1
```

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
超强CIFAR-100模型：融合EfficientNet和CNNCifar100的优点
集成所有最先进的优化技术
"""

import torch
import torch.nn as nn
import torch.nn.functional as F
import math
from torchvision import transforms


class SEBlock(nn.Module):
    """Squeeze-and-Excitation注意力机制"""
    def __init__(self, in_channels, reduction=16):
        super(SEBlock, self).__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Sequential(
            nn.Linear(in_channels, in_channels // reduction, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(in_channels // reduction, in_channels, bias=False),
            nn.Sigmoid()
        )

    def forward(self, x):
        b, c, _, _ = x.size()
        y = self.avg_pool(x).view(b, c)
        y = self.fc(y).view(b, c, 1, 1)
```

```
        return x * y.expand_as(x)


class StochasticDepth(nn.Module):
    """随机深度，提升泛化能力"""
    def __init__(self, drop_rate):
        super(StochasticDepth, self).__init__()
        self.drop_rate = drop_rate

    def forward(self, x, skip_connection):
        if not self.training or self.drop_rate == 0.:
            return x + skip_connection

        keep_prob = 1 - self.drop_rate
        random_tensor = keep_prob + torch.rand((x.shape[0], 1, 1, 1),
                                    dtype=x.dtype, device=x.device)
        binary_tensor = torch.floor(random_tensor)
        return (x / keep_prob) * binary_tensor + skip_connection


class EnhancedMBConvBlock(nn.Module):
    """增强的MBConv块，融合多种优化技术"""

    def __init__(self, in_channels, out_channels, stride, expand_ratio,
                 kernel_size=3, se_ratio=0.25, drop_rate=0.0):
        super(EnhancedMBConvBlock, self).__init__()

        self.stride = stride
        self.use_residual = stride == 1 and in_channels == out_channels

        # 扩展阶段
        expanded_channels = in_channels * expand_ratio
        self.expand_conv = None
        if expand_ratio != 1:
            self.expand_conv = nn.Sequential(
                nn.Conv2d(in_channels, expanded_channels, 1, bias=False),
                nn.BatchNorm2d(expanded_channels),
                nn.SiLU(inplace=True)
```

```
            )

        # 深度卷积
        self.depthwise_conv = nn.Sequential(
            nn.Conv2d(expanded_channels, expanded_channels, kernel_size,
                      stride=stride, padding=kernel_size//2, groups=expanded_chan
nels, bias=False),
            nn.BatchNorm2d(expanded_channels),
            nn.SiLU(inplace=True)
        )

        # SE注意力机制
        self.se_block = SEBlock(expanded_channels, reduction=int(1/se_ratio))
if se_ratio > 0 else None

        # 投影阶段
        self.project_conv = nn.Sequential(
            nn.Conv2d(expanded_channels, out_channels, 1, bias=False),
            nn.BatchNorm2d(out_channels)
        )

        # 随机深度
        self.stochastic_depth = StochasticDepth(drop_rate) if drop_rate > 0 els
e None

    def forward(self, x):
        identity = x

        # 扩展
        if self.expand_conv is not None:
            x = self.expand_conv(x)

        # 深度卷积
        x = self.depthwise_conv(x)

        # SE注意力
        if self.se_block is not None:
            x = self.se_block(x)
```

```
        # 投影
        x = self.project_conv(x)

        # 残差连接（带随机深度）
        if self.use_residual:
            if self.stochastic_depth is not None:
                x = self.stochastic_depth(x, identity)
            else:
                x = x + identity

        return x


class UltraEfficientNetCIFAR100(nn.Module):
    """超强CIFAR-100模型，融合所有优化技术"""

    def __init__(self, num_classes=100, dropout_rate=0.4, stochastic_depth_r
ate=0.2):
        super(UltraEfficientNetCIFAR100, self).__init__()

        self.dropout_rate = dropout_rate

        # 更强的Stem层
        self.stem = nn.Sequential(
            nn.Conv2d(3, 40, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(40),
            nn.SiLU(inplace=True)
        )

        # 深层架构配置 (参考您的CNNCifar100)
        # [expand_ratio, channels, num_blocks, stride, kernel_size, se_ratio]
        settings = [
            [1,  24,  1, 1, 3, 0.25],  # stage 1: 32×32 → 32×32
            [6,  32,  2, 2, 3, 0.25],  # stage 2: 32×32 → 16×16
            [6,  48,  3, 2, 5, 0.25],  # stage 3: 16×16 → 8×8
            [6,  96,  4, 2, 3, 0.25],  # stage 4: 8×8 → 4×4
            [6, 136,  4, 1, 5, 0.25],  # stage 5: 4×4 → 4×4
```

```
        [6, 232,  5, 2, 5, 0.25],  # stage 6: 4×4 → 2×2
        [6, 384,  2, 1, 3, 0.25],  # stage 7: 2×2 → 2×2
    ]

    # 构建网络层
    self.stages = nn.ModuleList()
    in_channels = 40
    total_blocks = sum([blocks for _, _, blocks, _, _, _ in settings])
    block_id = 0

    for expand_ratio, channels, num_blocks, stride, kernel_size, se_ratio in
settings:
        stage = nn.ModuleList()

        for i in range(num_blocks):
            block_stride = stride if i == 0 else 1
            # 计算随机深度率
            drop_rate = stochastic_depth_rate * block_id / total_blocks

            stage.append(EnhancedMBConvBlock(
                in_channels, channels, block_stride, expand_ratio,
                kernel_size, se_ratio, drop_rate
            ))
            in_channels = channels
            block_id += 1

        self.stages.append(stage)

    # 更强的头部
    self.head_conv = nn.Conv2d(384, 1536, kernel_size=1, bias=False)
    self.head_bn = nn.BatchNorm2d(1536)
    self.avgpool = nn.AdaptiveAvgPool2d(1)

    # 多层分类器（参考您的设计）
    self.classifier = nn.Sequential(
        nn.Dropout(dropout_rate),
        nn.Linear(1536, 768),
        nn.BatchNorm1d(768),
```

```
            nn.SiLU(inplace=True),
            nn.Dropout(dropout_rate * 0.7),
            nn.Linear(768, 384),
            nn.BatchNorm1d(384),
            nn.SiLU(inplace=True),
            nn.Dropout(dropout_rate * 0.5),
            nn.Linear(384, num_classes)
        )

        # 权重初始化
        self._initialize_weights()

    def _initialize_weights(self):
        """改进的权重初始化"""
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='r
elu')
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1)
                nn.init.constant_(m.bias, 0)
            elif isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, 0, 0.01)
                if m.bias is not None:
                    nn.init.constant_(m.bias, 0)

    def forward(self, x):
        # Stem
        x = self.stem(x)

        # 所有stage
        for stage in self.stages:
            for block in stage:
                x = block(x)

        # Head
        x = F.silu(self.head_bn(self.head_conv(x)))
        x = self.avgpool(x)
```

```
        x = torch.flatten(x, 1)

        # 分类器
        x = self.classifier(x)

        return F.log_softmax(x, dim=1)


def get_ultra_cifar100_model(dropout_rate=0.4):
    """获取超强CIFAR-100模型"""
    return UltraEfficientNetCIFAR100(
        num_classes=100,
        dropout_rate=dropout_rate,
        stochastic_depth_rate=0.2
    )


# 改进的数据增强策略
def get_ultra_transforms(is_training=True):
    """超强数据增强策略"""
    if is_training:
        return transforms.Compose([
            transforms.RandomCrop(32, padding=4),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomRotation(15),
            transforms.ColorJitter(
                brightness=0.4,
                contrast=0.4,
                saturation=0.4,
                hue=0.1
            ),
            transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.5071, 0.4867, 0.4408],
                std=[0.2675, 0.2565, 0.2761]
            ),
            transforms.RandomErasing(p=0.1, scale=(0.02, 0.33), ratio=(0.3, 3.
```

```
3))
        ])
    else:
        return transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.5071, 0.4867, 0.4408],
                std=[0.2675, 0.2565, 0.2761]
            )
        ])


# 改进的损失函数
class LabelSmoothingCrossEntropy(nn.Module):
    """标签平滑交叉熵损失"""
    def __init__(self, epsilon=0.1, reduction='mean'):
        super(LabelSmoothingCrossEntropy, self).__init__()
        self.epsilon = epsilon
        self.reduction = reduction

    def forward(self, input, target):
        log_prob = F.log_softmax(input, dim=-1)
        weight = input.new_ones(input.size()) * self.epsilon / (input.size(-1) -
1.)
        weight.scatter_(-1, target.unsqueeze(-1), (1. - self.epsilon))
        loss = (-weight * log_prob).sum(dim=-1)

        if self.reduction == 'mean':
            return loss.mean()
        elif self.reduction == 'sum':
            return loss.sum()
        return loss


if __name__ == "__main__":
    # 测试模型
    model = get_ultra_cifar100_model()
    x = torch.randn(2, 3, 32, 32)
```

```
    output = model(x)
    print(f"Model output shape: {output.shape}")
    print(f"Model parameters: {sum(p.numel() for p in model.parameters() if
p.requires_grad):,}")
```

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
轻量级Ultra CIFAR-100模型：优化版本，减少复杂度，提升训练速度
"""

import torch
import torch.nn as nn
import torch.nn.functional as F


class LightweightSEBlock(nn.Module):
    """轻量级SE注意力机制"""
    def __init__(self, in_channels, reduction=16):
        super(LightweightSEBlock, self).__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Sequential(
            nn.Linear(in_channels, max(in_channels // reduction, 8), bias=Fals
e),
            nn.ReLU(inplace=True),
            nn.Linear(max(in_channels // reduction, 8), in_channels, bias=Fals
e),
            nn.Sigmoid()
        )

    def forward(self, x):
        b, c, _, _ = x.size()
        y = self.avg_pool(x).view(b, c)
        y = self.fc(y).view(b, c, 1, 1)
        return x * y
```

```
class LightweightMBConvBlock(nn.Module):
    """轻量级MBConv块"""

    def __init__(self, in_channels, out_channels, stride, expand_ratio,
                 kernel_size=3, se_ratio=0.25):
        super(LightweightMBConvBlock, self).__init__()

        self.stride = stride
        self.use_residual = stride == 1 and in_channels == out_channels

        # 扩展阶段
        expanded_channels = in_channels * expand_ratio
        self.expand_conv = None
        if expand_ratio != 1:
            self.expand_conv = nn.Sequential(
                nn.Conv2d(in_channels, expanded_channels, 1, bias=False),
                nn.BatchNorm2d(expanded_channels),
                nn.ReLU6(inplace=True)  # 使用ReLU6替代SiLU，速度更快
            )

        # 深度卷积
        self.depthwise_conv = nn.Sequential(
            nn.Conv2d(expanded_channels, expanded_channels, kernel_size,
                      stride=stride, padding=kernel_size//2, groups=expanded_chan
nels, bias=False),
            nn.BatchNorm2d(expanded_channels),
            nn.ReLU6(inplace=True)
        )

        # 轻量级SE注意力
        self.se_block = LightweightSEBlock(expanded_channels, reduction=1
6) if se_ratio > 0 else None

        # 投影阶段
        self.project_conv = nn.Sequential(
            nn.Conv2d(expanded_channels, out_channels, 1, bias=False),
            nn.BatchNorm2d(out_channels)
```

```
        )

    def forward(self, x):
        identity = x

        # 扩展
        if self.expand_conv is not None:
            x = self.expand_conv(x)

        # 深度卷积
        x = self.depthwise_conv(x)

        # SE注意力
        if self.se_block is not None:
            x = self.se_block(x)

        # 投影
        x = self.project_conv(x)

        # 残差连接
        if self.use_residual:
            x = x + identity

        return x


class LightweightUltraCIFAR100(nn.Module):
    """轻量级Ultra CIFAR-100模型，优化训练速度"""

    def __init__(self, num_classes=100, dropout_rate=0.3):
        super(LightweightUltraCIFAR100, self).__init__()

        self.dropout_rate = dropout_rate

        # 简化的Stem层
        self.stem = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(32),
```

```
            nn.ReLU6(inplace=True)
        )

        # 简化的架构配置 - 减少块数量
        # [expand_ratio, channels, num_blocks, stride, kernel_size, se_ratio]
        settings = [
            [1,  16,  1, 1, 3, 0.25],  # stage 1: 32×32 → 32×32
            [6,  24,  2, 2, 3, 0.25],  # stage 2: 32×32 → 16×16
            [6,  40,  2, 2, 5, 0.25],  # stage 3: 16×16 → 8×8
            [6,  80,  3, 2, 3, 0.25],  # stage 4: 8×8 → 4×4
            [6, 112,  2, 1, 5, 0.25],  # stage 5: 4×4 → 4×4
            [6, 192,  2, 2, 5, 0.25],  # stage 6: 4×4 → 2×2
        ]

        # 构建网络层
        self.stages = nn.ModuleList()
        in_channels = 32

        for expand_ratio, channels, num_blocks, stride, kernel_size, se_ratio in
settings:
            stage = nn.ModuleList()

            for i in range(num_blocks):
                block_stride = stride if i == 0 else 1

                stage.append(LightweightMBConvBlock(
                    in_channels, channels, block_stride, expand_ratio,
                    kernel_size, se_ratio
                ))
                in_channels = channels

            self.stages.append(stage)

        # 简化的头部
        self.head_conv = nn.Conv2d(192, 1280, kernel_size=1, bias=False)
        self.head_bn = nn.BatchNorm2d(1280)
        self.avgpool = nn.AdaptiveAvgPool2d(1)
```

```
        # 简化的分类器
        self.classifier = nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(1280, 512),
            nn.ReLU6(inplace=True),
            nn.Dropout(dropout_rate * 0.7),
            nn.Linear(512, num_classes)
        )

        # 权重初始化
        self._initialize_weights()

    def _initialize_weights(self):
        """权重初始化"""
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='r
elu')
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1)
                nn.init.constant_(m.bias, 0)
            elif isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, 0, 0.01)
                if m.bias is not None:
                    nn.init.constant_(m.bias, 0)

    def forward(self, x):
        # Stem
        x = self.stem(x)

        # 所有stage
        for stage in self.stages:
            for block in stage:
                x = block(x)

        # Head
        x = F.relu6(self.head_bn(self.head_conv(x)))
        x = self.avgpool(x)
```

```
        x = torch.flatten(x, 1)

        # 分类器
        x = self.classifier(x)

        return F.log_softmax(x, dim=1)


def get_lightweight_ultra_cifar100_model(dropout_rate=0.3):
    """获取轻量级Ultra CIFAR-100模型"""
    return LightweightUltraCIFAR100(
        num_classes=100,
        dropout_rate=dropout_rate
    )


if __name__ == "__main__":
    # 测试模型
    model = get_lightweight_ultra_cifar100_model()
    x = torch.randn(2, 3, 32, 32)
    output = model(x)
    print(f"Model output shape: {output.shape}")
    print(f"Model parameters: {sum(p.numel() for p in model.parameters() if
p.requires_grad):,}")
```

```
python federated_cifar100_main.py --model lightweight_ultra --epochs 350
--num_users 20 --frac 0.5 --local_ep 5 --local_bs 64 --lr 0.01 --optimizer s
gd --momentum 0.9 --weight_decay 5e-4 --dropout_rate 0.3 --label_smoot
hing 0.1 --alpha 0.3 --gpu 0 --iid 1
```

2.non-iid

（1）普通FedAvg

```
python federated_main.py \
    --dataset cifar100 \
```

```
    --model cnn \
    --epochs 350 \
    --lr 0.035 \
    --num_users 40 \
    --frac 0.5 \
    --local_ep 8 \
    --local_bs 64 \
    --optimizer sgd \
    --iid 0 \
    --num_classes 100 \
    --gpu 0 \
    --verbose 1
```

（2）优化FedAvg

```
python federated_cifar100_main.py --model lightweight_ultra --epochs 200 --num_users 20 --frac 0.5 --local_ep 5 --local_bs 64 --lr 0.001 --optimizer adamw --weight_decay 1e-3 --dropout_rate 0.3 --label_smoothing 0.1 --alpha 0.3 --gpu 0 --iid 0
```